

AdvR 13 – Quasiquotation

Motivation

Why Quasiquotation?

Quotation = capturing code without evaluating it. **Unquotation** = selectively evaluating parts of quoted code.
Together: **quasiquotation**.

```
cement <- function(...) {  
  args <- ensyms(...)  
  paste(purrr::map(args, as_string), collapse = " ")  
}  
cement(Good, morning, Hadley)
```

```
#> [1] "Good morning Hadley"
```

Problem: how to use a **variable** inside a quoting function?

```
cement(Good, time, name) # quotes literally
```

```
#> [1] "Good time name"
```

!! to the Rescue

!! (bang-bang) **unquotes** — tells the quoting function to evaluate:

```
name <- "Hadley"; time <- "morning"  
cement(Good, !!time, !!name)
```

```
#> [1] "Good morning Hadley"
```

Compare:

```
paste("Good", time, name) # paste evaluates -> quote where needed  
cement(Good, !!time, !!name) # cement quotes -> unquote where needed
```

Quoted vs. Evaluated Arguments

- **Evaluated** argument: obeys normal R evaluation rules
- **Quoted** argument: captured by the function, processed in a custom way

Test: run the argument **outside** the function. If it fails, it's quoted:

```
library(MASS)    # works: MASS is quoted  
MASS             # fails: not a regular object
```

```
#> Error: object 'MASS' not found
```

Exercises (Motivation)

- ❶ For each function below, which arguments are quoted and which evaluated?

```
library(MASS)
mtcars2 <- subset(mtcars, cyl == 4)
with(mtcars2, sum(vs))
sum(mtcars2$am)
rm(mtcars2)
```

1

Call	Quoted	Evaluated
<code>library(MASS)</code>	<code>MASS</code>	—
<code>subset(mtcars, cyl == 4)</code>	<code>cyl == 4</code>	<code>mtcars</code>
<code>with(mtcars2, sum(vs))</code>	<code>sum(vs)</code>	<code>mtcars2</code>
<code>sum(mtcars2\$am)</code>	<code>am (by \$)</code>	<code>mtcars2\$am</code>
<code>rm(mtcars2)</code>	<code>mtcars2</code>	—

Quoting

Capturing Expressions

Four rlang quoting functions:

```
expr(x + y)           # capture YOUR code
```

```
#> x + y
```

```
f2 <- function(x) enexpr(x)
f2(a + b + c)          # capture CALLER's code
```

```
#> a + b + c
```

```
f <- function(...) enexprs(...)
f(x = 1, y = 10 * z)   # capture all ... args
```

```
#> $x
```

```
#> [1] 1
```

```
#>
```

```
#> ... [output truncated]
```

Summary:

	Developer	User
One	expr()	enexpr()
Many	exprs()	enexprs()

Capturing Symbols

`ensym()` / `ensyms()` — like `enexpr()` but **only** accept symbols or strings:

```
f <- function(...) ensyms(...)
f(x)
```

```
#> [[1]]
```

```
#> x
```

```
f("x")
```

```
#> [[1]]
```

```
#> x
```

Useful when you want to restrict input to variable names.

rlang	Base R
<code>expr()</code>	<code>quote()</code>
<code>enexpr()</code>	<code>substitute()</code>
<code>exprs()</code>	<code>alist()</code>
<code>enexprs()</code>	<code>as.list(substitute(...()))</code>

Key difference: base R functions **don't support unquoting** (!!).

substitute() Does Substitution

`substitute()` both **quotes** and **substitutes** values:

```
f4 <- function(x) substitute(x * 2)
f4(a + b + c)
```

```
#> (a + b + c) * 2
```

Use the second argument to make substitution explicit:

```
substitute(x * y * z, list(x = 10, y = quote(a + b)))
```

```
#> 10 * (a + b) * z
```

Exercises (Quoting)

- ① Compare f1 and f2. What does each return when called with f(a + b, c + d)?

```
f1 <- function(x, y) exprs(x = x, y = y)
f2 <- function(x, y) enexprs(x = x, y = y)
```

Solutions (Quoting)

- ① f1 uses `exprs()` (developer quoting) — returns the literal symbols `x` and `y`. f2 uses `enexprs()` (user quoting) — captures the caller's expressions:

```
f1 <- function(x, y) exprs(x = x, y = y)
f2 <- function(x, y) enexprs(x = x, y = y)
str(f1(a + b, c + d))
```

```
#> List of 2
#> $ x: symbol x
#> $ y: symbol y
```

```
str(f2(a + b, c + d))
```

```
#> List of 2
#> $ x: language a + b
#> $ y: language c + d
```

Unquoting

!! — Unquoting One Argument

!! evaluates a single expression and inlines the result into the AST:

```
x <- expr(-1)
expr(f(!!x, y))
```

```
#> f(-1, y)
```

Works with symbols, constants, and calls:

```
a <- sym("y"); b <- 1
expr(f(!!a, !!b))
```

```
#> f(y, 1)
```


!! Preserves Precedence

Because !! works with ASTs, not strings, operator precedence is correct:

```
x1 <- expr(x + 1)
x2 <- expr(x + 2)
expr (!!x1 / !!x2)
```

```
#> (x + 1)/(x + 2)
```

Result: $(x + 1) / (x + 2)$. If we pasted strings, we'd get $x + 1 / x + 2$ — wrong precedence.

Unquoting a Function

To unquote in the **function position**, use extra parentheses:

```
f <- expr(pkg::foo)
expr((!!f)(x, y))
```

```
#> pkg::foo(x, y)
```

Or use `call2()` for clarity:

```
call2(f, expr(x), expr(y))
```

```
#> pkg::foo(x, y)
```

!!! — Unquoting Many Arguments

!!! (bang-bang-bang) **splices** a list into a call:

```
xs <- exprs(1, a, -b)
expr(f(!!!xs, y))
```

```
#> f(1, a, -b, y)
```

```
ys <- set_names(xs, c("a", "b", "c"))
expr(f(!!!ys, d = 4))
```

```
#> f(a = 1, b = a, c = -b, d = 4)
```

Works in any rlang function that takes ..., including `call2()`:

```
call2("f", !!!xs, expr(y))
```

```
#> f(1, a, -b, y)
```

The Polite Fiction of !!

From R's perspective, !! is just double negation:

```
!!TRUE
```

```
#> [1] TRUE
```

```
!!!TRUE
```

```
#> [1] FALSE
```

!! and !!! behave as real operators **only inside rlang quoting functions**. Outside, you can get silent errors:

```
df <- data.frame(x = 1:5)
y <- 100
with(df, x + !!y)  # !!100 = 1, not 100!
```

```
#> [1] 2 3 4 5 6
```

Exercises (Unquoting)

- ① Given these components, use `!!` and `!!!` to construct the calls below:

```
xy <- expr(x + y); xz <- expr(x + z)
yz <- expr(y + z); abc <- exprs(a, b, c)

(x + y) / (y + z)           # use !! twice
sum(a, b, c)                 # use !!!
mean(c(a, b, c), na.rm = TRUE) # use !!!
```

Solutions (Unquoting)

1

```
xy <- expr(x + y); xz <- expr(x + z)
yz <- expr(y + z); abc <- exprs(a, b, c)
expr(!!xy / !!yz)
```

```
#> (x + y)/(y + z)
```

```
expr(sum(!!!abc))
```

```
#> sum(a, b, c)
```

```
expr(mean(c(!!!abc), na.rm = TRUE))
```

```
#> mean(c(a, b, c), na.rm = TRUE)
```

Non-quoting and Tidy Dots

Base R Non-quoting Techniques

Base R disables quoting rather than using `!!`:

Technique	Example
Quoting/non-quoting pair	<code>\$</code> (quoted) vs <code>[[</code> (evaluated)
Character argument	<code>rm(list = "x")</code> instead of <code>rm(x)</code>
Switch argument	<code>library(pkg, character.only = TRUE)</code>
Eval-on-fail	<code>help(var)</code> tries eval, then quotes

```
x <- list(var = 1, y = 2); var <- "y"  
c(x$var, x[[var]]) # $ quotes; [[ evaluates
```

```
#> [1] 1 2
```


!!! with ... (Tidy Dots)

Problem 1: elements already in a list:

```
dfs <- list(  
  a = data.frame(x = 1, y = 2),  
  b = data.frame(x = 3, y = 4))  
dplyr::bind_rows(!!!dfs)
```

```
#>   x y  
#> 1 1 2  
#> 2 3 4  
#> ... [output truncated]
```

Problem 2: dynamic argument names — use `:=`:

```
var <- "x"; val <- c(4, 3, 9)  
tibble::tibble(!!var := val)
```

```
#> # A tibble: 3 x 1  
#>       x  
#>   <dbl>  
#> ... [output truncated]
```

`:=` is needed because R's grammar doesn't allow `!!var = val`.

exec() and do.call()

`rlang::exec()` calls a function with `!!!` and `:=` support:

```
params <- list(na.rm = TRUE, trim = 0.1)
exec("mean", x = 1:10, !!!params)
```

```
#> [1] 5.5
```

Base R equivalent: `do.call()`:

```
do.call("mean", c(list(x = 1:10), params))
```

```
#> [1] 5.5
```

`exec()` also supports dynamic argument names:

```
arg_name <- "na.rm"
exec("mean", 1:10, !!arg_name := TRUE)
```

```
#> [1] 5.5
```

Exercises (Tidy Dots)

① One way to implement `exec()`:

```
exec <- function(f, ..., .env = caller_env()) {  
  args <- list2(...)  
  do.call(f, args, envir = .env)  
}
```

Describe how it works. What are the key ideas?

- 1 `list2(...)` collects arguments with tidy dots support (!!! for splicing, := for dynamic names). Then `do.call()` calls `f` with those arguments in the specified environment. Key ideas: (a) `list2()` gives any function tidy dot semantics, (b) `do.call()` provides the actual function call, (c) `.env` ensures correct scoping.